Open in app ↗

Medium Search Write

Get unlimited access to the best of Medium for less than $1/week. Become a member

Data Science and MLOps with Feast: Mastering Feature Store



Oleh Dubetcky Follow 8 min read · Mar 31, 2024

43 2

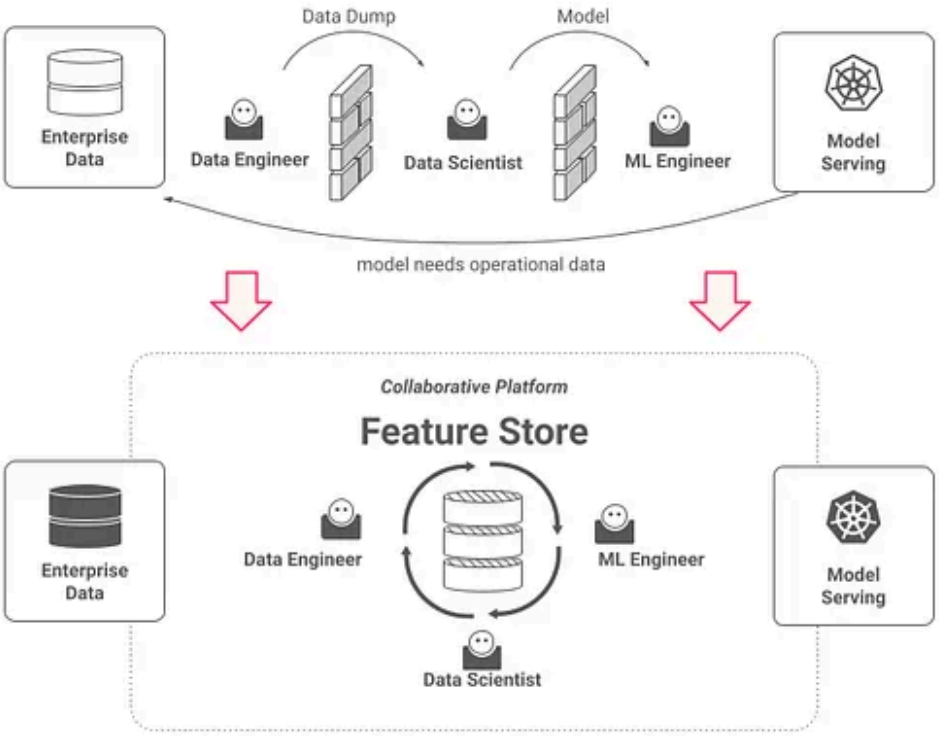
Machine learning (ML) has transformed the operational landscape of businesses and their approach to data-informed decision-making. Nonetheless, a key hurdle in ML pipelines lies in the efficient management and provisioning of features. Feature stores emerge as a solution to this challenge. These centralized repositories empower organizations to store, explore, distribute, and oversee features pivotal to ML models. Let's consider feature store, delving into their pivotal role across all stages of the machine learning lifecycle.



Photo by Mario Losereit on Unsplash

What is a Feature Store?

A feature store is a centralized repository designed to manage and serve features for machine learning (ML) models efficiently. Features, in the context of ML, refer to the individual measurable properties or characteristics of the data used to train and make predictions with ML models. These features can range from simple attributes like age or gender to more complex engineered features derived from raw data.



Feature Store (Source: https://www.featurestore.org/)

The primary functions of a feature store include:

1. **Storage:** Storing a wide range of features in a structured format, making them easily accessible for model training and serving.
2. **Discovery:** Providing mechanisms for discovering and exploring available features, including their metadata and statistics.
3. **Sharing:** Enabling collaboration by allowing teams to share and reuse features across different ML projects and pipelines.
4. **Management:** Facilitating the organization and management of features, including versioning, tracking changes, and ensuring consistency and quality.
5. **Serving:** Efficiently serving features to ML models during both training and inference stages, ensuring real-time or batch processing as needed.

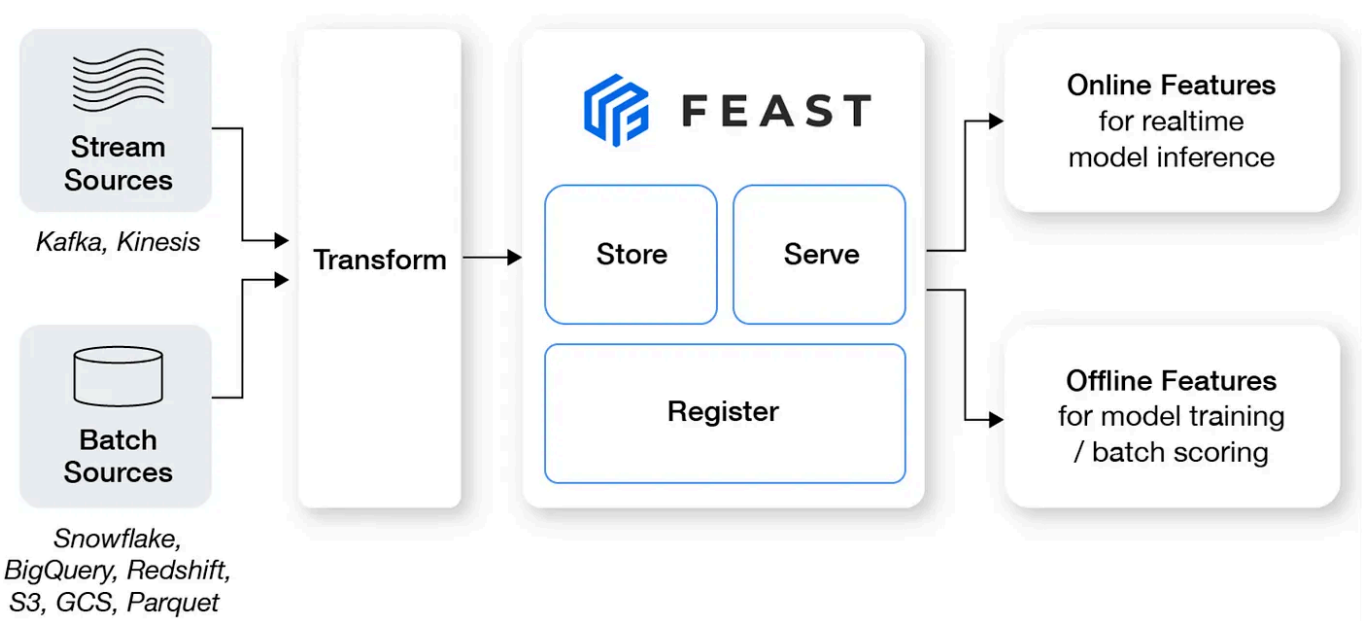
Feature stores address several common challenges in ML pipelines, such as:

- **Data Consistency:** Ensuring that features used for model training are consistent across different stages of the ML pipeline, preventing data skew and improving model accuracy.
- **Feature Engineering:** Streamlining the process of feature engineering by providing a centralized platform for feature creation, transformation, and reuse.
- **Scalability:** Handling large volumes of feature data and serving requests efficiently, even in high-throughput production environments.

By providi`ng a unified platform for feature management and serving, feature stores contribute to the scalability, reproducibility, and efficiency of ML workflows, ultimately accelerating the development and deployment of ML models in production settings.

Feature Store Architecture

Feast has a well-defined architecture designed to manage and serve features efficiently. Here's an overview of the key components in the Feast architecture:



Feast Core:

- **Metadata Store:** This component stores metadata about features, feature sets, entities, and related information. It provides a central repository for managing and organizing features.
- **Feature Registry:** The feature registry is a service that allows users to define, register, and version features and feature sets. It ensures consistency and versioning across different feature definitions.
- **REST API:** Feast Core exposes a REST API for interacting with the metadata store and feature registry. This API enables users to register features, retrieve metadata, and perform administrative tasks programmatically.

Feast Serving:

- Online Serving: Feast provides an online serving component that serves real-time feature values to machine learning models during inference. It

supports high-throughput, low-latency requests and can scale horizontally to handle large workloads.

- Batch Serving: Batch serving allows users to retrieve feature values in bulk for offline model training or batch inference. It provides an interface for querying historical feature data based on timestamps or time ranges.

Data Storage Backend:

Feast supports various data storage backends, including relational databases (e.g., MySQL, PostgreSQL), NoSQL databases (e.g., Bigtable, DynamoDB), and cloud storage services (e.g., Google Cloud Storage, Amazon S3). These backends are used to store feature data persistently and can scale to accommodate large volumes of data.

Feature Ingestion:

Feature ingestion pipelines extract feature data from raw data sources, transform it into a format suitable for storage and serving, and load it into the data storage backend. Feast provides connectors and integrations for ingesting data from databases, data lakes, streaming platforms, and other sources.

Feature Retrieval:

Feast SDKs and client libraries allow developers and data scientists to retrieve feature values from the serving layer programmatically. These SDKs provide an easy-to-use interface for accessing features in both online and batch serving modes.

CLI and UI Tools:

Feast includes command-line interface (CLI) tools and graphical user interface (UI) tools for managing and interacting with the feature store. These tools provide functionality for registering features, querying metadata, inspecting feature sets, and performing administrative tasks.

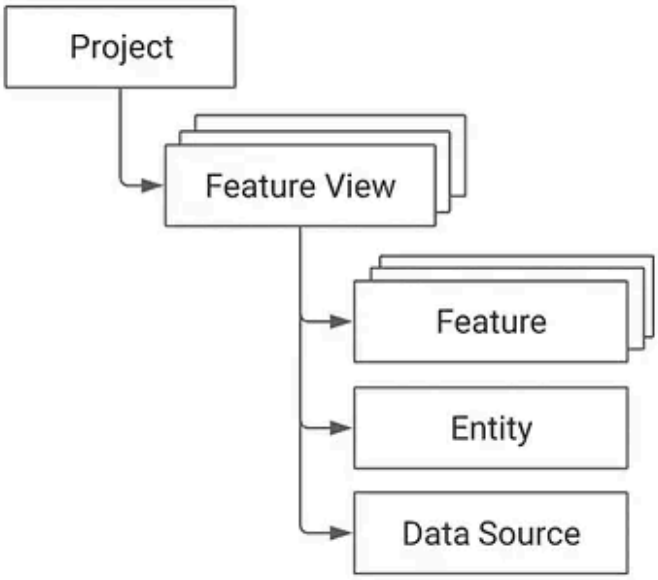
Security and Authentication:

Feast supports authentication and authorization mechanisms to secure access to the feature store and ensure data privacy and integrity. It integrates with identity providers, authentication services, and access control systems to enforce security policies and permissions.

By leveraging these components, Feast provides a scalable, reliable, and feature-rich platform for managing and serving features in machine learning workflows. Its modular architecture allows for flexibility and extensibility, enabling users to customize and integrate Feast into their existing infrastructure and workflows seamlessly.

Key concepts in Feast

Feast implements hierarchical feature storage to help you store and manage features. Feast feature repository has the following structure:



https://docs.feast.dev/getting-started/concepts/overview

Here's what each of the components in a Feast feature repo means:

· **Project.** A project is a collection of related features and their data sources. Projects are isolated from each other, and you can't reuse features from one project in another. Projects were supported to ensure backward compatibility with previous versions of Feast. The concept of projects might change as Feast developers simplify the framework.

· **Feature view.** A feature view is a group of feature data from a specific data source. Feature views allow you to consistently define features and their data sources, enabling the reuse of feature groups across a project. If your features are stored in more than one location, you can specify a feature view for each location and later join all the features together. Feature views make the addition of new features to your existing data very easy as well — as you gather new groups of features, you can create separate feature views for them and then merge them with your old data.

· **Data source**. In Feast, each feature view has a data source. A data source is where the raw feature data is stored, like a local `.parquet` file or a GCP (Google Cloud Platform) bucket. You can have as many data sources as you want, but you can't mix different types of sources together.

· **Feature service**. A feature service is an object that contains features from one or more feature views. You can use feature services to create logically related groups of feature views.

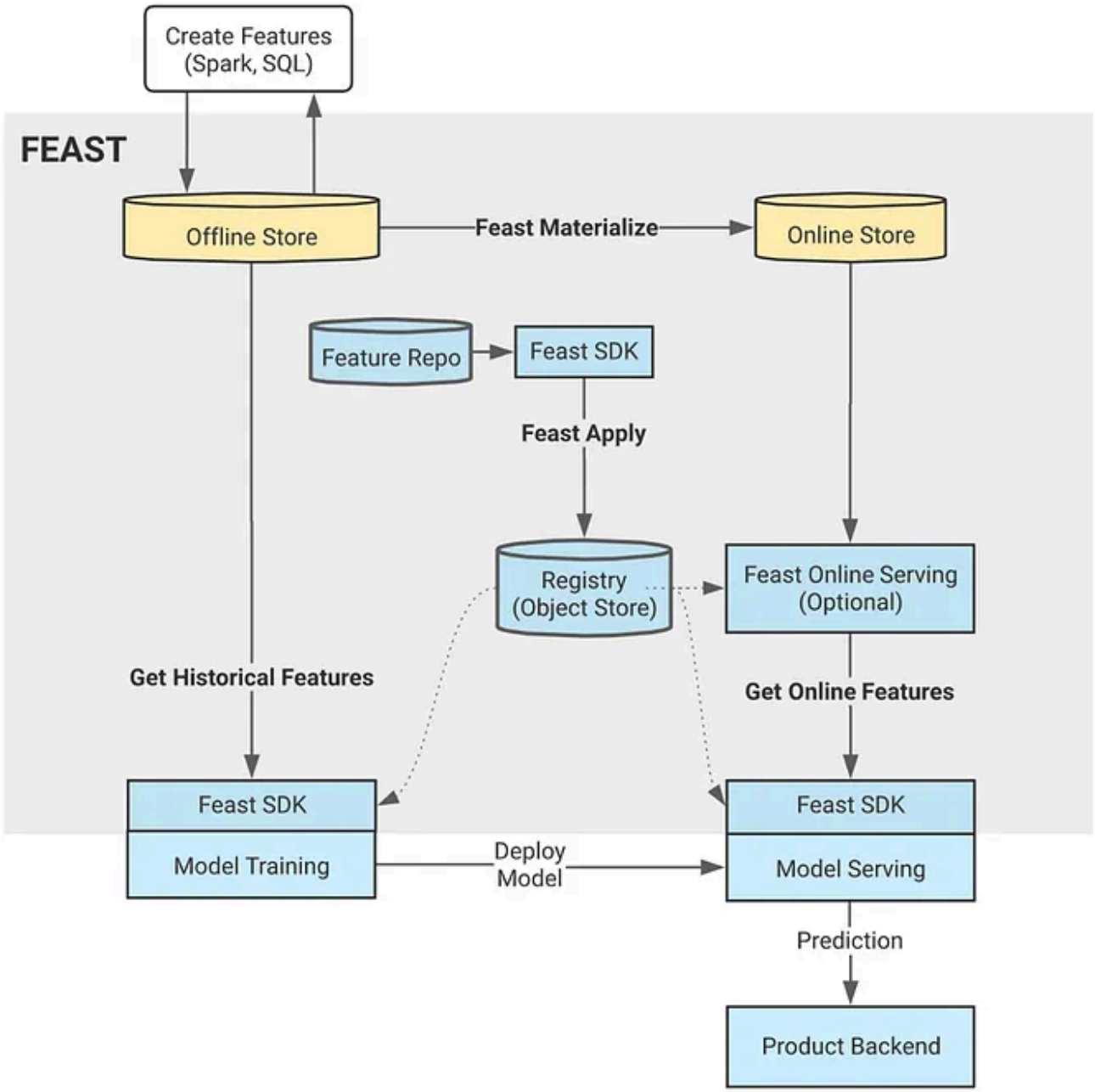
· **Entity**. The Feast docs describe entities as a collection of semantically related features. In practical terms, entities can be the individuals or objects that your feature data relates to. As an example, if you have a pneumonia dataset, you can set your patients as your entities and assign unique IDs to them for identification. You can then use the IDs to store and retrieve specific feature values. Feast also uses entities to correctly join data from different feature views.

· **Timestamps**. Feast uses timestamps to ensure that features from different sources are joined in the correct chronological order. Primarily, this is so that you can avoid using very old data for training or prediction.

· **Dataset**. A dataset is a group of feature views and entities. Feast datasets allow engineers to combine data from different feature views for analysis and training.

Feast infrastructure

Feast's concepts are pretty simple, but how do they work in practice? To help you understand this, let's take a look at this diagram from the Feast docs:



https://docs.feast.dev/getting-started/architecture-and-components/overview

This diagram displays the architecture of a Feast project. Before we describe how Feast handles data, let's first understand the concepts in the diagram:

· **Offline store.** The offline store is where you store your features. In the terminology of Feast, the offline store contains historical features that you can use for analysis or training. Feast can both retrieve and write data to the

offline store. This type of store is “offline” because it is located outside a Feast environment.

· **Online store.** The online store is where Feast stores features for low-latency access. The online store is designed to accelerate inference.

· **Feature repo.** A feature repository contains the definitions of a Feast feature store. It defines where features are stored, how they should be retrieved, and what they contain. If your offline store is on your local machine, a feature repository can also contain the raw feature data.

· **Registry.** A registry is a catalog of feature definitions and their metadata. It defines the infrastructure of your feature repository and where feature data comes from.

· **Provider.** A provider is the implementation of a specific offline or online feature store. Feast has specialized providers for AWS, GCP, and local environments.

With these components in mind, here’s how you use Feast at a high level:

· First, you define your entities, feature views, feature services, and data sources and register them in your feature store. Feast will register feature and data source definitions in your feature repository’s registry.

· Then, you use your feature views to fetch feature data from your offline stores (data sources). In the language of Feast, this is called historical retrieval. You can join different feature views together to create a dataset that you can then analyze, save, or use for training.

· After you train and deploy a model, you can fetch (materialize) the latest feature values from the offline store for inference. When you materialize features, they are stored in the online store for performance reasons.

· As you add new features to your offline stores, you can continuously materialize them to keep your online store fresh. Additionally, if necessary, you can do historical retrieval of your new and old features, join them all together, and retrain your model on the new dataset.

Sample use-case tutorials

Feast provides various sample use-case tutorials to help users understand how to leverage the feature store for machine learning projects effectively. Here are some common sample use-case tutorials you might find with Feast:

- **Driver Ranking:** This tutorial demonstrates building a model to predict driver completion of trips using Feast for feature management.
- **Fraud Detection on GCP:** This example showcases using Feast with Google Cloud Platform (GCP) for fraud detection in a machine learning pipeline.
- **Real-time Credit Scoring on AWS:** This tutorial demonstrates building a real-time credit scoring system leveraging Feast on Amazon Web Services (AWS).
- **Driver Stats on Snowflake:** This example guides you through using Feast with Snowflake to manage features for driver analytics.

Each of these sample use-case tutorials provides step-by-step instructions, code examples, and best practices for building and deploying feature stores with Feast. They demonstrate how Feast can be used to solve real-world

machine learning challenges and accelerate the development and deployment of machine learning models in production environments.

If you liked the article, you can support the author by clapping below 👏 Thanks for reading!

Oleh Dubetcky|Linkedin

Feast Feature Store Data Science Mlops



Written by Oleh Dubetcky

298 followers · 404 following

Management consultant delivering integrated Cybersecurity & IT solutions, aligning business goals with secure, high-performing tech strategies.

Follow

Responses (2)



Baolongnguyen Mac

What are your thoughts?



Daniel García
Oct 17, 2024

Very interesting!

1 Reply



Kitwana Ezechiel
Apr 29

Good article to read right now.

Reply

More from Oleh Dubetcky



Oleh Dubetcky

Low-Level Design: A Comprehensive Guide

Low-Level Design (LLD) is a crucial phase in software development that bridges the gap...

Feb 24 1

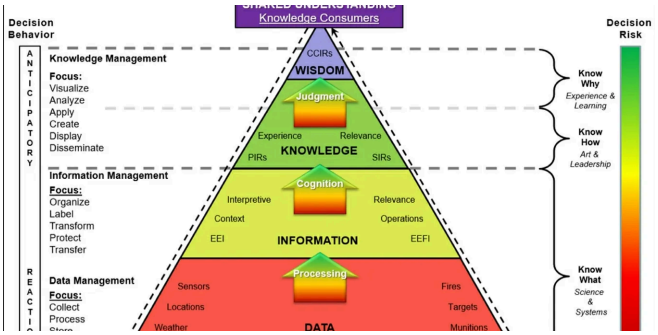


Oleh Dubetcky

Mastering LLM Observability: A Hands-On Guide to Langfuse and...

Large language models (LLMs) are revolutionizing applications, from chatbots t...

Mar 2 2

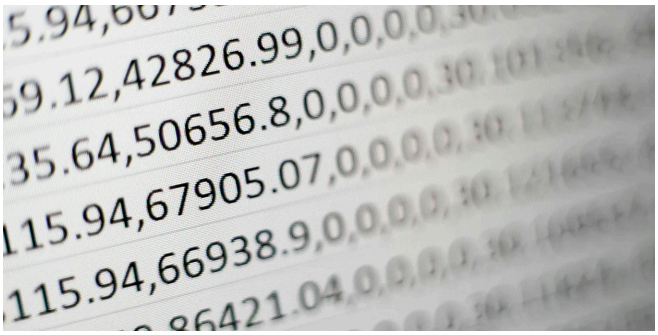


Oleh Dubetcky

Project Management for Data Science: KDD, SEMMA and CRISP...

Managing data science projects involves navigating complexity through tailored...

Jul 15, 2024 11



Oleh Dubetcky

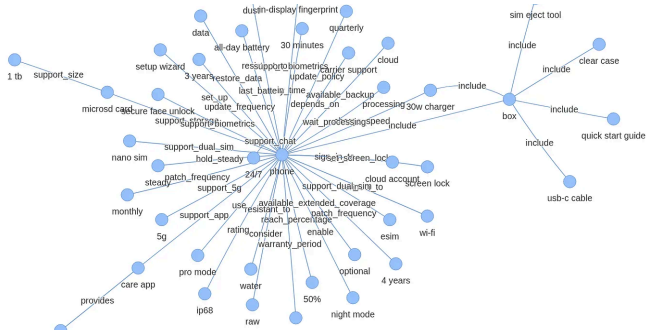
Detecting Data Drift with Evidently AI

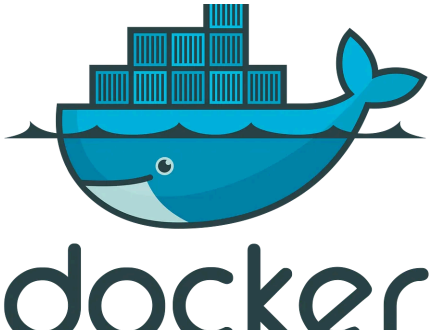
Detecting data drift with Evidently AI is straightforward and essential for ensuring...

Nov 4, 2024 67

See all from Oleh Dubetcky

Recommended from Medium





In The Muse Junction by Asutosh Nayak

How to Build a QA System using Knowledge Graph and LLM

Use LLMs to build a QA system on top of Knowledge Graphs

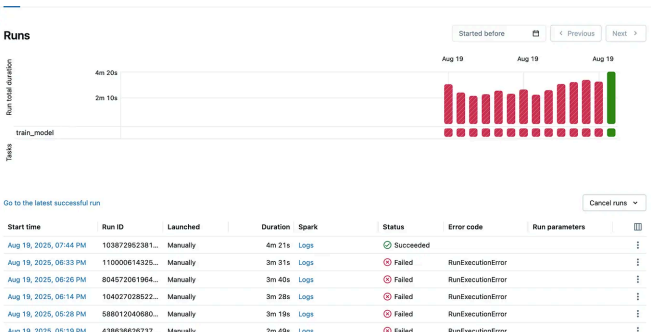
6d ago 75

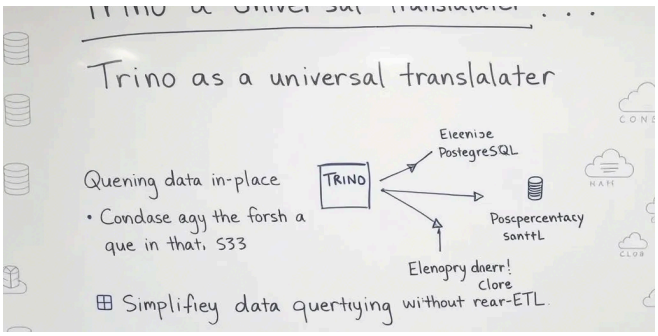
Abhinav

Docker Is Dead—And It's About Time

Docker changed the game when it launched in 2013, making containers accessible and...

Jun 9 6K 162





Claudia Yao

End-to-End MLOps Workflow with Databricks + MLflow

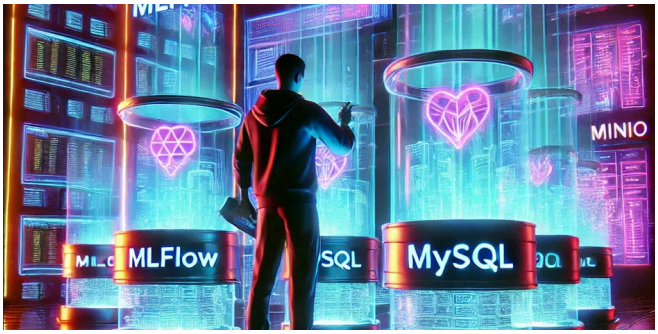
Recently, I came across an online tutorial series led by two MLOps professionals. They...

In EndToEndData by Prem Vishnoi(cloudvala)

The Complete Guide to Installing and Running Trino: From Zero to...

Why Trino Will Change How You Think About Data Querying

Aug 20 14



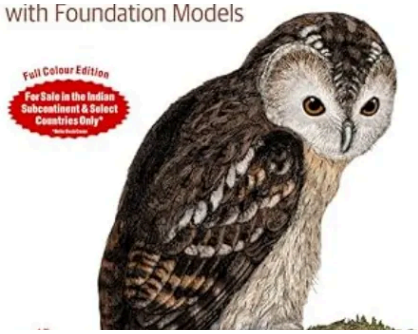
Hitoruna

Remote MLFLow setup with Docker

Or how to setup a remote MLFlow server easily

Mar 28 51

Jun 11 82



In Android Alchemy by Prakash Sharma

Stop watching random AI courses: Complete Guide to AI Engineering

Ultimate guide for AI Engineering preparation , Six months preparation guide —

Jul 31 876 22

See more recommendations